PROGRAMMABLE LOGIC DEVICES

CHAPTER 11

In the previous chapter we introduced the basics of digital electronics – gates and combinational logic, flip-flops and sequential logic – and we illustrated their application with some examples: a modulo-*n* counter, a multiplexed LED display, and an *n*-pulse generator. In that chapter we used primarily *standard logic*; that is, small blocks of logic (gates, flip-flops, counters, registers) packaged as single integrated circuits.

However, as we remarked often, there is an alternative (and usually better) way to implement those kinds of circuits, namely, the use of *programmable logic devices* (PLDs).[1] A PLD consists of a chip with lots of logic (gates and registers, and sometimes much more), in which the *connections* are programmable. But it's not a computer program: in a computer the program tells the processor *what to do*; in a PLD the program tells the chip *how to connect* its component pieces.

11.1 A brief history

The first PLDs (1975) were the Signetics' "Integrated Fuse Logic" devices, with a handful of uncommitted gates and a matrix of fuses (literally!) that could be selectively blown to leave the desired interconnections. You "programmed" the things with a chart of fuse links, or (later) by designating the fuse locations manually on a computer screen. In subsequent developments, Signetics added flip-flops to the mix, enabling sequential circuits; and Monolithic Memories devised a simplified family known as PALs (programmable array logic), with a more usable text-entry design language called PALASM. By the middle of the 1980s we had Lattice's GALs (generic array logic), which used electrically reprogrammable memory and a chameleon-like output cell that could be either a flip-flop or a gate. There were improved programming languages, too, namely CUPL and ABEL; these, and their successors, are all generically called *HDLs* (hardware description languages).

Around the same time Xilinx introduced the FPGA (field-programmable gate array), an alternative programmable logic scheme with a "finer-grained" architecture – many more gates and registers, organized as a fabric of logic blocks, surrounded by I/O blocks, with more flexible interconnections – and whose configuration programming was held off-chip in a separate small non-volatile memory device (a serial "configuration ROM"), and loaded into on-chip (and volatile) SRAM at power-up.

FPGAs and complex PLDs, improving nicely with time, are both part of the essential toolkit of circuit designers. Contemporary cPLDs (the "c" stands for "complex") have from 32 to 2000 macrocells, with tens of thousands of gates, and operate at clock speeds up to a few hundred megahertz; they use on-chip nonvolatile program memory, and can be programmed and reprogrammed in-circuit, using simple serial protocols (e.g., JTAG). There are variants[2] with essentially zero quiescent current. They have highly predictable timing.

Contemporary FPGAs are denser, topping out at around a million flip-flops, in packages (see Figure 11.1) with as many as 1738 pins (!). They may include blocks with dedicated memory, arithmetic modules (ALUs, MACs, other DSP), interface (USB, Ethernet, PCI), and other specialized functions. With such extensive capabilities, it's now routine to program in standard designs for large modules such as a video processor, PCIe controller, Bluetooth, or even a complete microprocessor (and peripherals), with plenty of programmable logic left over to form a "system-on-a-chip." These standard programmed ("soft") designs are known as "IP" (intellectual property, some of which you may have to license); if you put in an IP microprocessor, it is called a *soft processor* core.[3] An efficient alternative, if you want a processor inside your FPGA, is the use of a hybrid FPGA, with the microprocessor and peripherals already hardwired.[4]

[1] Officially "PLDs," but generally meant to include cPLDs (complex PLDs) and FPGAs (field-programmable gate arrays).

[2] Notably the Xilinx (originally Philips) CoolRunner™ series, and the Lattice ispMACH™4000Z series.

[3] Examples are the Xilinx MicroBlaze, the Altera Nios-II, the Lattice Mico, and the Actel ARM.

[4] Examples are the PowerPC (in Xilinx FPGAs), the AVR (in Atmel's FPSLIC), and the ARM (in Altera's FPGAs).

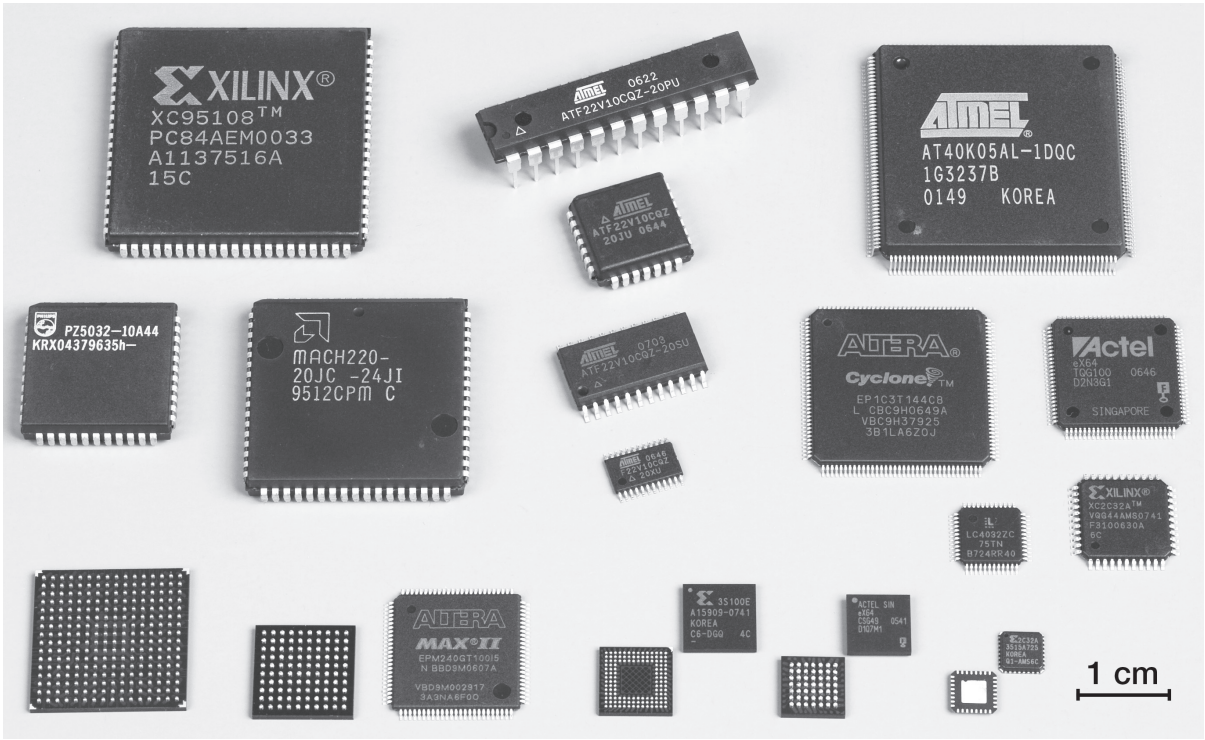


Figure 11.1. A selection of programmable logic packages. At upper left are three PLCCs (84, 68, and 44 pins), to be compared with their denser QFP brethren at upper right (going clockwise: PQFP-208, TQFP-100, VQFP-44, TQFP-48, and TQFP-144). Down the middle are displayed the same sPLD type (22V10) in four increasingly dense packages: DIP-24, PLCC-28, SOIC-24, and TSSOP-24. Across the bottom are the densest packages (left to right): FTBGA-256, FBGA-100 (and the same IC in a TQFP-100), BGA-132, BGA-49 (CSP-49), and QFN-32; for the last three both top and underside are shown.

For such complex devices, the legacy programming tools (CUPL, ABEL) that evolved with small PLDs are hopeless.[5] Current practice favors either graphical schematic entry, or one of the two contemporary and powerful text-based HDLs (called *Verilog* and *VHDL*); presently we'll illustrate both. For most digital designs, programmable logic is generally a better choice than standard logic, because (a) one chip replaces many – less wiring, smaller finished product, less inventory, lower cost; and (b) it is easy to reprogram the part if the initial design is flawed, or if you want to add features.

11.2 The hardware

Contemporary PLDs are fiendishly complex… and they're not getting any simpler. High-end devices already have upwards of 5 billion transistors (and counting). To make it all understandable, we'll take it in easy stages: first the classic simple PAL architectures (e.g., 22V10) and the more flexible PLAs (e.g., Xilinx CoolRunner-II), and then the complex and register-rich FPGAs.

11.2.1 The basic PAL

A good starting point is the basic PAL, an example of a simple PLD. The classic device is the 22V10, the part number signifying 10 output *macrocells* (more on this soon), among a total of 22 inputs and outputs (originally offered in a 24-pin package). The PAL circuit scheme consists of a programmable array of connections, whereby both inputs and feedback outputs can be selectively connected to a set of many-input AND gates; the outputs from a fixed number of such AND gates then feed an OR gate. Its output can be used directly (a combinational output), or it can feed a type-D flip-flop (a "registered" output); the circuitry for these output options is called an "output macrocell." Figure

[5] You *can* still get (from Atmel or Lattice) the classic small "sPLDs" (*simple* PLDs): 16V8, 20V8, 22V10, and 26V12, for which CUPL and ABEL are completely adequate.

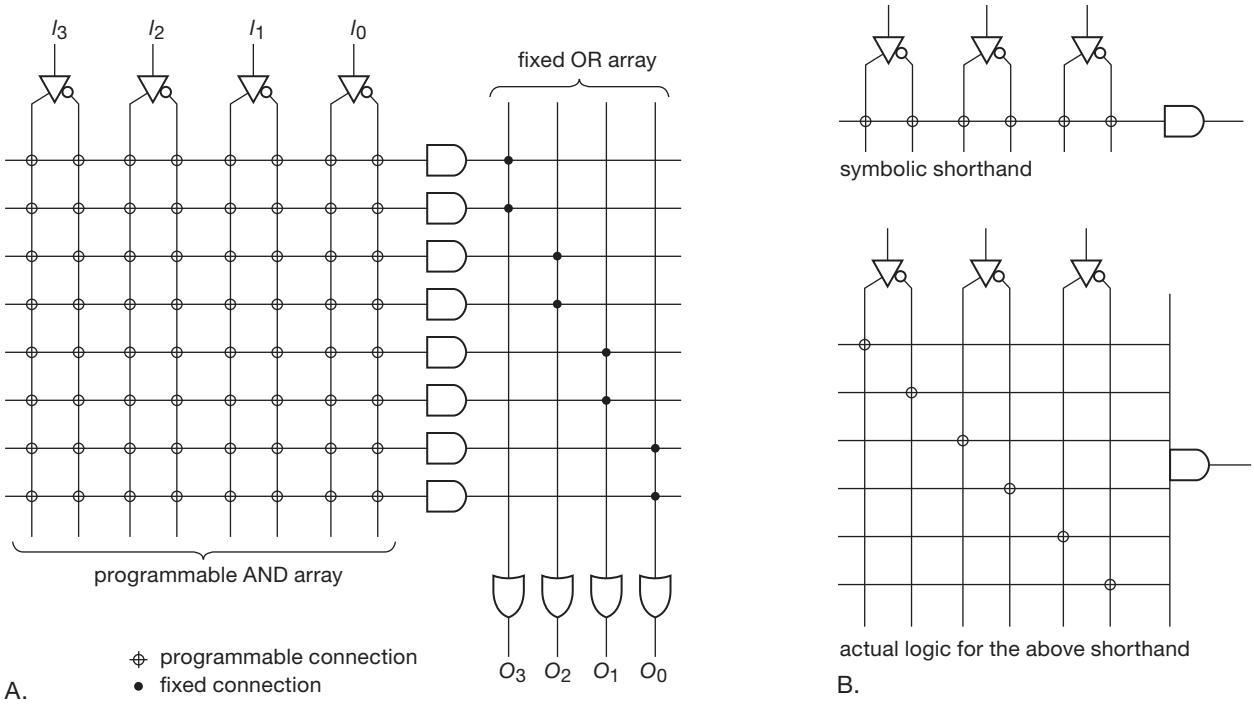


Figure 11.2. A. In a "PAL" (Programmed Array Logic) every available logic signal (or its complement) can be connected to many-input ANDs; the outputs of several such ANDs are OR-ed to form the logical outputs, each then passing through an output macrocell on the way to an output pin. B. Each AND gate has many inputs, shown in conventional shorthand and in fully expanded form.

11.2 shows the basic idea, in a simplified scheme with only four input signals; here each logic output comes from an OR gate fed from two of the 8-input ANDs, and that output would drive the output macrocell. (Note the shorthand notation used in Figure 11.2A for the many-input AND-gates and OR gates, shown expanded in Figure 11.2B.)

In real life a PAL has many more inputs and gates. Figure 11.3 shows the actual scheme for the 22V10, which, though impressive looking, is in fact tiny by contemporary standards. The 12 inputs (and their complements), along with feedback from the 10 outputs (and their complements) – thus 44 signals total – are all brought into the connection matrix, in which any set of them can be connected to any 44-input AND, whose output is then OR-ed with another 7 to 15 ANDs (more toward the center of the chip, fewer toward the ends) to create an output (one of ten). That output does not go directly off the chip, but rather gets massaged in an output macrocell (Figure 11.4), which consists of a bit of programmable logic that lets you latch the OR output in a flip-flop, or just pass it through without latching. You can see from Figure 11.4 that the output, whether "registered" or combinational, can be true or complemented, and further can be three-stated.

The 22V10 is adequate for many logic tasks: you can make a shift register, or a counter, or an address decoder, or just a collection of what's called "random logic." But by contemporary standards it is extremely limited, both by the small number of registers and I/O pins, and also by the constraint of a common CLK signal; you cannot, for example, make a ripple counter.

One evolutionary path has been the creation of what we might call a "super-PAL," in which the macrocells allow individual clocking from the logic array, and in which many more macrocells are packed into one IC. These often include schemes for expanding the logic by sharing product terms across macrocells. Such extensions of the basic PAL architecture are called "cPLDs" (complex PLDs), and are widely available.[6] They range in capacity up to 2000 or more macrocells (some of which may be "buried," i.e., available for internal interconnections but not connected to I/O pins), and they allow asynchronous clocking.

The "programmable-AND/fixed-OR" architecture

[6] For example, the Altera MAX7000 series, the Lattice Mach 4000 series, or the Xilinx CoolRunner-II series (the latter uses the more advanced PLA architecture; see §11.2.2).

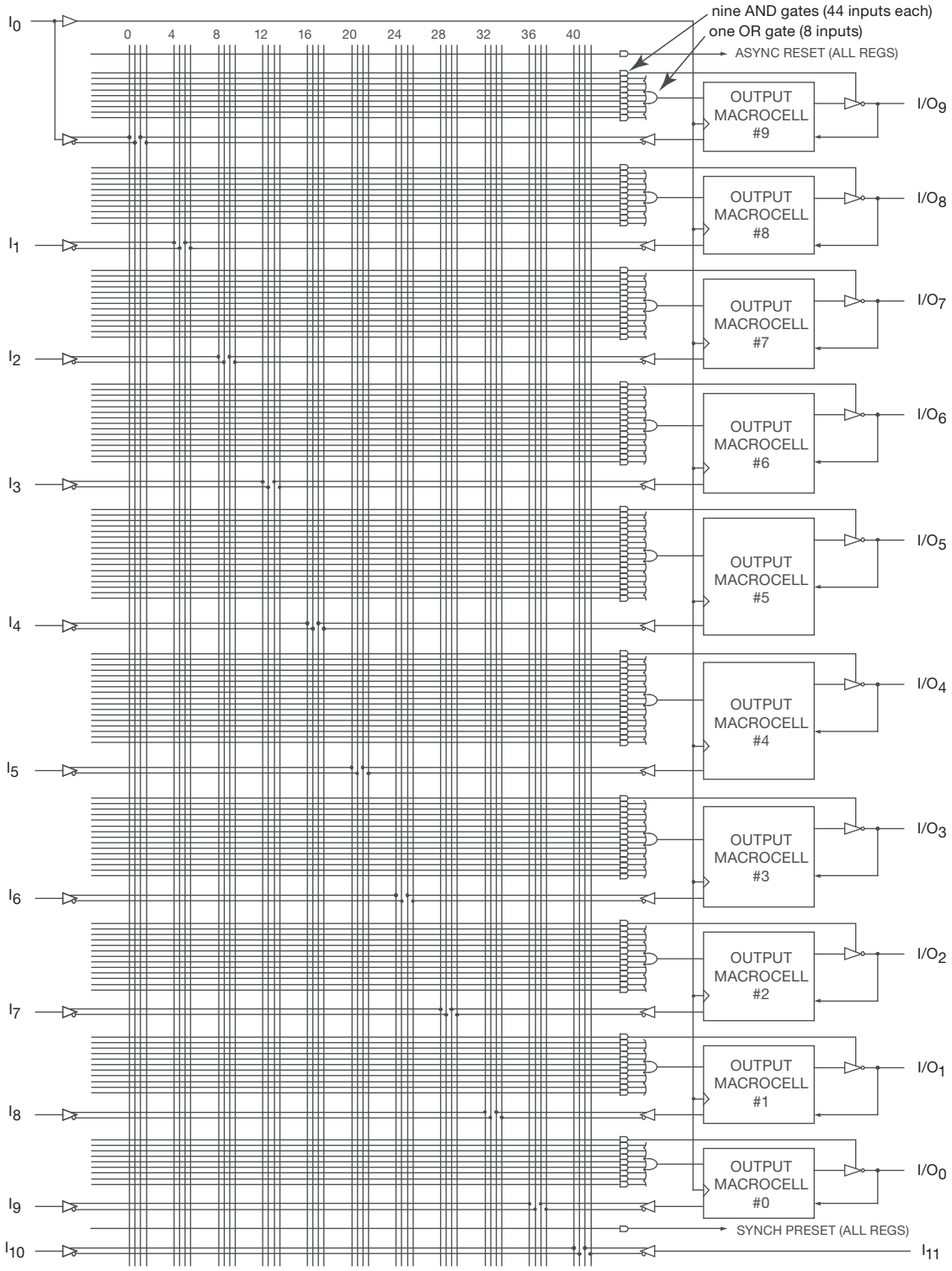


Figure 11.3. Logic array of the classic 22V10 PAL: programmable-AND/fixed-OR. The output macrocell can be programmed as a type-D flip-flop or a simple pass-through; see Figure 11.4. (Adapted from the Lattice ispGAL22V10 datasheet, ©Lattice Semiconductor Corporation, 2004.)

(sometimes called "sum of products") can be a limitation also, particularly when you want to implement a complicated bit of combinational logic, because it forces the implementation to be a single sum (OR) of several products (ANDs). The next step up in logical complexity is the programmed logic array (PLA).

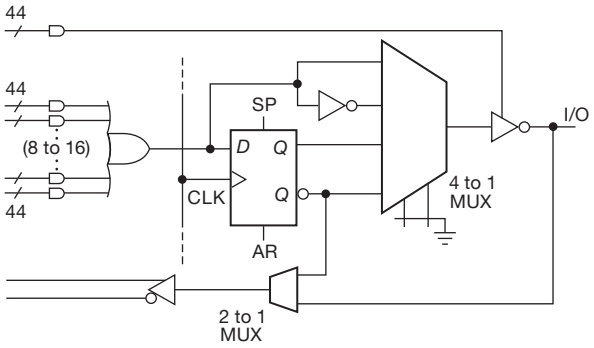


Figure 11.4. Programmable "macrocell" of the classic 22V10 PAL: true or inverted, registered or combinational, with three-state output and feedback into the logic array. The CLK, SP (synch preset), and AR (asynch reset) are common to all 10 macrocells, whereas a "product term" (a 44-input AND) is available for each macrocell's three-state control.

11.2.2 The PLA

In the PLA, the logic array consists of programmable ANDs feeding *programmable* (as opposed to fixed) ORs (Figure 11.5). Manufacturers don't generally distinguish these parts by calling them PLAs; they just call them cPLDs (same as the large PALs); you have to look in the datasheet to see what's really inside. A contemporary example is the Xilinx CoolRunner-II (XC2C) series of cPLDs, currently offered with 32 to 512 macrocells (the latter half buried, given its modest 324-pin package).

As we'll see soon, the software you use to program these parts pretty much hides their inner architecture from the user; you just write a bunch of HDL code, run the software, and see (a) if it fits, and (b) if it meets the speed and timing-delay requirements.

11.2.3 The FPGA

The PALs, PLAs, and cPLDs above are all examples of what might be called "logic-rich–register-poor" architectures. After all, with even the lowly 22V10 you have a dozen 44-input AND gates feeding each OR-gated macrocell – but only a handful (OK, *two* handfuls) of flip-flops.

A complementary approach is provided by FPGAs, which consist of a fabric of flip-flops, each fed from a modest bit of logic (typically just a four-input lookup table, or LUT), embedded in a vast array of interconnecting wiring. Here we're talking tens of thousands to millions of these cells, called variously "configurable logic blocks" (CLBs), or "logic elements" (LEs); and, these are often accompanied by blocks of memory (RAM), arithmetic functions such as multipliers or embedded processors, and interface elements such as PCIe or Ethernet. Such FPGA architectures are well characterized as "logic-poor–register-rich," or as "fine grained."[7] Figure 11.6 shows a typical logic cell, in this case from Altera's Cyclone-II FPGA family.

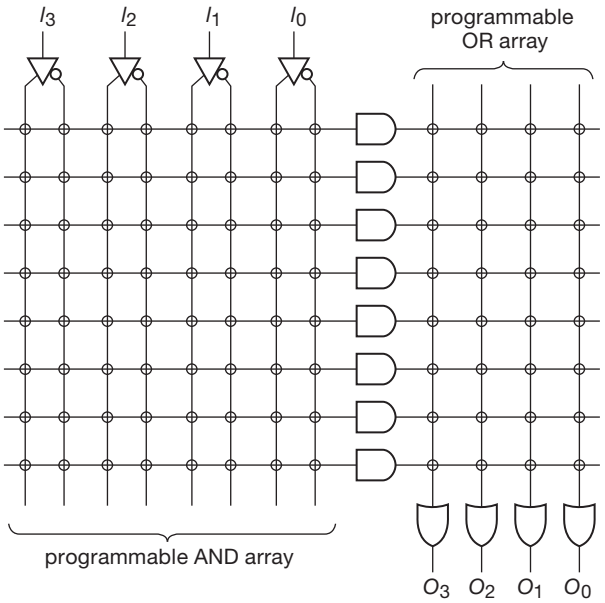


Figure 11.5. A PLA (Programmed Logic Array) is more flexible than the PAL (Figure 11.2), by allowing programming of both the AND and OR arrays.

You can think of FPGAs as highly advanced custom ICs that are configured by the user. The high-end FPGAs easily accommodate what you would normally think of as separate microprocessors and peripherals – a configurable "system-on-a-chip." And FPGA manufacturers compete vigorously to put the latest process technologies into their products. As a result, their products increasingly appeal to manufacturers as an alternative to full-custom ICs: the

[7] As luck would have it, this neat organization is challenged by some of the cPLDs that are actually chimeras, or FPGAs in disguise, for example the Altera Max-II cPLDs (which use 4-input LUTs), Cypress Delta39K cPLDs, or the Lattice MachXO "crossover cPLDs."

latter[8] involve a costly design cycle, provide less flexibility, and often delay the introduction of a new product into the marketplace. "Time-to-market" is critical in consumer electronics, making FPGAs cost effective, in spite of their higher per-unit costs. An additional benefit is the ability to upgrade their internal circuitry (for example to fix bugs, add signal processing options, or whatever).

Typically you'll pay a dollar or two for cPLDs and for small FPGAs, ten dollars or so for intermediate size FPGAs, and from hundreds of dollars up to a thousand dollars or more (!) for FPGAs of the largest size and highest performance. Traditionally FPGAs have not been low-power devices, with typical quiescent currents of the order of milliamps to tens of milliamps (and additional dynamic power dissipation, of course, when clocked rapidly). However, micropower FPGAs are becoming available, aimed squarely at portable applications: an example is the Actel IGLOO series, with quiescent power dissipation in the 5–50 μW range.

The big players in the high-end FPGA market are Altera and Xilinx (the originator), with smaller FPGAs from Actel, Atmel, Cypress, and Lattice.

11.2.4 The configuration memory

A programmable logic device has to have its program held *somewhere.* Traditionally, sPLDs and cPLDs have always kept their configuration on-chip, in non-volatile storage: the earliest PLDs (e.g., the original MMI PALs) used one-time on-chip fusible links, but they were quickly superseded by reprogrammable CMOS memory. Some early devices (e.g., the original Altera Max 7000) used UV-erasable (EPROM) memory (with a quartz window, so you could blast it with the appropriate amnesia-inducing dose – typically of 20 minutes duration – of ultraviolet light), but quickly enough the industry adopted *electrically*-erasable memory (EEPROM, or "flash" memory). That is how all sPLDs and cPLDs are now made; they can be erased in seconds, and reprogrammed typically in less than a minute. There are specifications for the minimum *data-retention* time of the on-chip memory (typically 20 years), and for its *endurance* (typically a minimum of 1000 to 10,000 program–erase cycles).

Early sPLDs had to be programmed in a *device programmer* (which you bought from a company such as BP Microsystems, Needhams Electronics, or DataIO), then placed in the target circuit. This worked OK for through-hole (DIP) packaged parts, for which you could provide sockets in the final resting place; but it was awkward for soldered-in-place surface-mount parts, because reprogramming required removal of the IC. The solution is "in-system" programmability, which was retrofitted onto some sPLDs (e.g., Lattice's ispGAL22V10, an ISP version of the standard 22V10). All contemporary cPLDs are in-system programmable, using a simple serial data protocol such as JTAG (IEEE 1149) boundary scan.[9] You connect to the serial programming pins with a "pod," connected to the host computer that holds the programming data via a USB connection. Pods typically cost around $100; see the discussion of JTAG in §14.7.4.

For *FPGAs* the situation is somewhat different. Traditionally, FPGAs had no on-chip ROM; instead they inhaled their configuration data at power-up from an external serial "configuration ROM," then held the configuration on-chip in static (volatile) memory. Although this is still the protocol for many FPGAs, there are now available some FPGA families with on-chip nonvolatile flash memory, for example from Actel, Lattice, and Xilinx. As with contemporary cPLDs, programming is done in-circuit, via a comparably priced pod.

11.2.5 Other programmable logic devices

It's worth remembering that a simple digital memory is "programmable logic": it produces a stored *n*-bit output for each possible *m*-bit (address) input. That is how it is used in the general state machine of Figure 10.64, and, at a simpler level, a small (e.g., 16×1) lookup table is used in the logic elements of an FPGA.

There are recent developments in the form of *mixed-signal* programmable logic; an example is the Cypress PSoC™ mixed-signal programmable family, which incorporates amplifiers, filters, and other analog components on a programmable chip that includes a microprocessor.

11.2.6 The software

To use programmable logic devices, you've got to learn some software tools. You begin with design *entry*, either with a hardware description language (HDL) or with schematic entry. Then you run a *simulation* tool, to verify that the design does what you intend. Next is *synthesis*, which converts the design to a "netlist," which describes the logical connections. The netlist is then fitted to the target device, a process called *place and route*. Finally,

[8] Sometimes called an *ASIC*, for application-specific integrated circuit.

[9] The IEEE has developed an industry standard data format for using JTAG for in-system configuration, known as IEEE 1532.

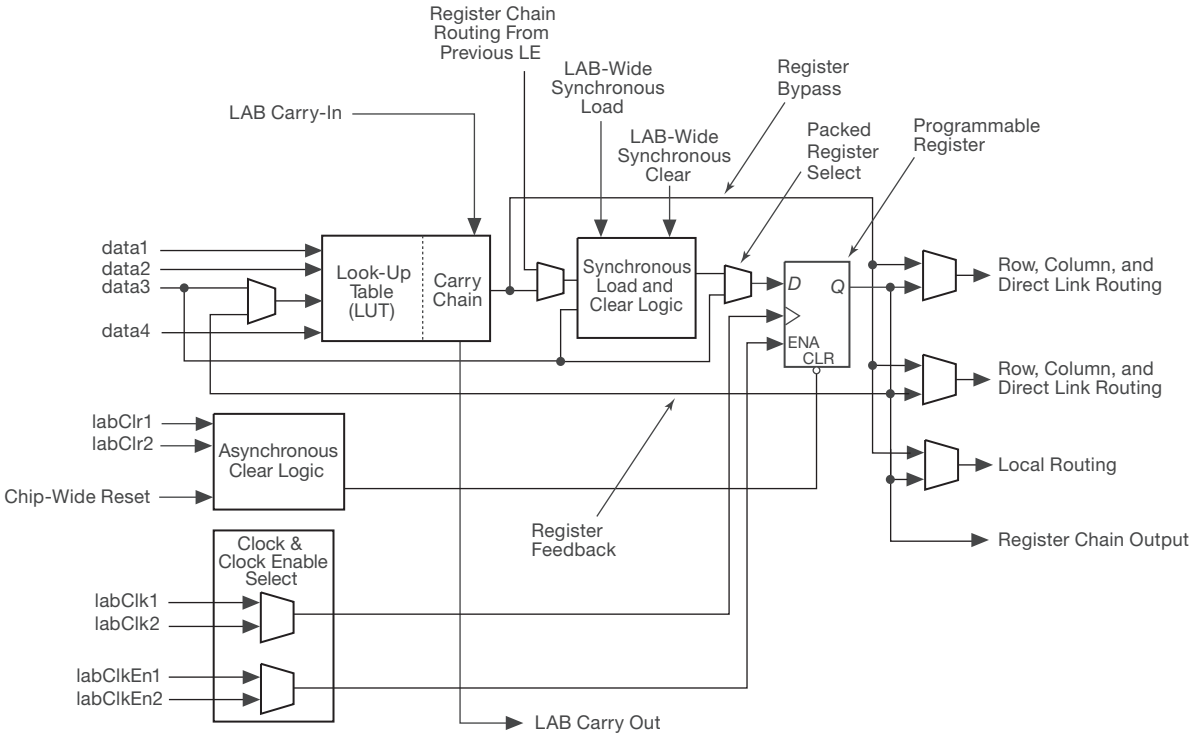


Figure 11.6. An FPGA logic element (LE), one of some tens of thousands in a typical FPGA, from the Altera "Cyclone II" family. The clocked flip-flop and lookup table (LUT) are decorated with extra logic and control signals from the larger logic array block (LAB), for flexible expansion. (Adapted from Altera Cyclone II Device Family datasheet, ©Altera Corporation, 2007.)

the fitted design is downloaded to the programmable logic chip, either to the chip itself (if it has on-chip non-volatile memory), or to a configuration ROM (from which it loads at power-up), or possibly hot-loaded from a processor directly to the powered-up chip.

FPGA manufacturers typically provide free software for their low-to-mid-range products,[10] but they may charge from several hundred dollars to several thousand dollars for software that supports their high-end offerings; some of the cost of the latter is attributable to the included software "cores" (code modules that creates blocks of useful hardware, sometimes called "IP," for intellectual property). There may be deep discounts offered to educational and nonprofit users.

These software tools are not terribly easy to learn, and they have a distressing tendency to change with time. They are often afflicted with bugs, requiring skillful workarounds. It can be a full-time job keeping up. It's always helpful, therefore, to have some company (which misery loves); that is, to use the same software tools as those the vastly more knowledgeable people around you are using.

We will not attempt to teach these tools. However, to illustrate the general process we next show an example of a complete circuit design done five different ways (§11.3), one of which is with a cPLD (§11.3.3).

11.3 An example: pseudorandom byte generator

The quickest way to see how a circuit idea can be implemented in several ways is, well, simply to see an example. Ours is what might be described as the ultimate gibberish machine,[11] namely a circuit that sends out a succession of pseudorandom bytes, as standard RS-232 serial data. In this section we begin with a *block diagram*, then we show the circuit implemented first in (a) *standard logic*, then in (b) *programmable logic* (with entry in two popular methods, namely graphic *schematic entry*, and text-based

[10] An example being the Xilinx "WebPACK," which you get directly over the Internet (if you have the patience to download and install the >5 GByte set of files!).

[11] Or the proverbial monkey-at-a-typewriter.

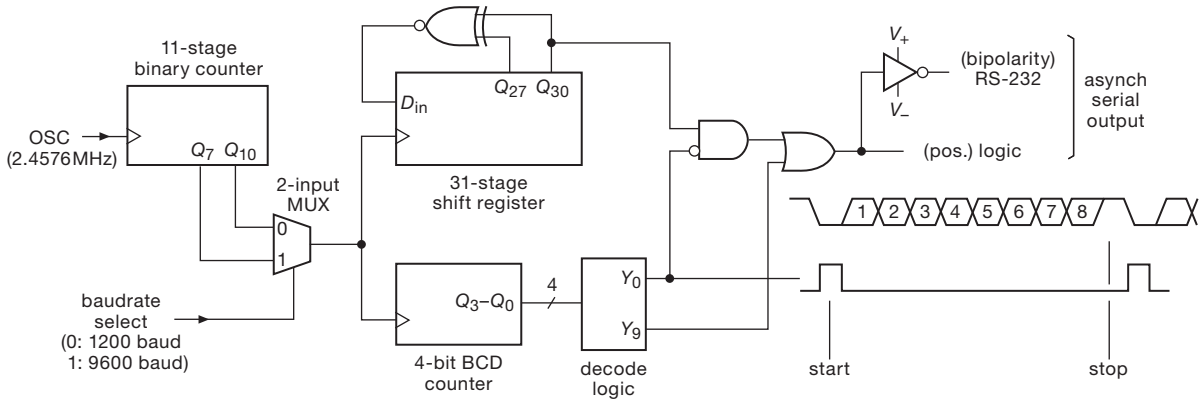


Figure 11.7. Block diagram of RS-232 pseudorandom byte generator. A 2-input MUX (multiplexer) selects the baudrate from a subdivided oscillator; the resulting clock shifts a maximal-length linear feedback shift register to produce pseudorandom bits, which are framed with START and STOP bits to produce 8n1-formatted serial bytes.

hardware description language.[12] Finally, we show how the same function is easily implemented with a *microcontroller*, the inexpensive (and indispensable) programmable computer-on-a-chip (complete with internal memory and I/O functions), intended for "embedded" use in any electronic device. Embedded microcontrollers are the subject of Chapter 15 – but we couldn't resist showing them now, simply because they are *the* attractive design alternative.

We wrap up this section with some advice – in particular, which hardware to use, paired with their appropriate design tools, and the pros and cons of each.

11.3.1 How to make pseudorandom bytes

Section 13.14 describes a simple method to produce a "pseudorandom" bit sequence (PRBS), using a shift register whose serial input comes from an XOR of two (or more) downstream bits.[13] The sequence so produced is *pseudo*random because, though it has many properties of randomness, it is not truly random; in fact, it is completely deterministic, with a repeat interval (with properly chosen XORed bits) of $2^n - 1$ for a register of n stages.[14]

Our example uses a register of length 31 (Figure 11.7), for which XOR feedback from bits 28 and 31 produces a maximal-length sequence. Its length ($2^{31} - 1$, or 2,147,483,647) corresponds to a repeat interval of 2.6 days at the maximum serial output rate of 9600 baud.[15] The shift register is clocked at a selectable rate of either 1200 or 9600 shifts/second, with the output bits grouped into 8-bit bytes, each of which is framed by a "START" and "STOP" bit pair. This is known as *asynchronous serial* communication, typified by the familiar RS-232C serial computer port. This circuit allows selection of baud rate (1200 or 9600), and formats the output as 8-bit data, no parity, and one stop bit (this is usually abbreviated "8n1").

As explained in §14.7.8, the actual voltage levels for serial port communication, described in the RS-232C specification, are (annoyingly) of both polarities, with ± 5 V to ± 15 V levels corresponding to the two logic states; to add a bit of confusion, those outputs are logically inverted, with logic HIGH corresponding to the negative signaling level, and vice versa.[16] The good news is that there are plenty of RS-232C driver–receiver interface chips available, most of which include internal charge-pump voltage generators so that you can power them from a single positive logic supply (usually $+3.3$ V or $+5$ V); an example is the MAX3232 shown in Figure 11.8.

[12] In particular, *Verilog* and *VHDL*, the two popular choices.

[13] Sometimes called a *linear feedback shift register*, or LFSR, and introduced earlier in §8.12.4A.

[14] It should be noted that there are far better computational algorithms for generating pseudorandom bit sequences; this method was chosen for its abject simplicity.

[15] Readers unhappy with the concept that a random sequence eventually *repeats* might prefer something a bit longer, say a shift register of length 71 (with XOR tap at 65), whose repeat interval is approximately 8 billion years (about half the age of the universe). *Real* purists can elect a longer register, say of length 167 (XOR tap at 161), which won't repeat for 10^{38} years (about 10^{28} times the age of the universe).

[16] Sometimes called *MARK* and *SPACE*, respectively.

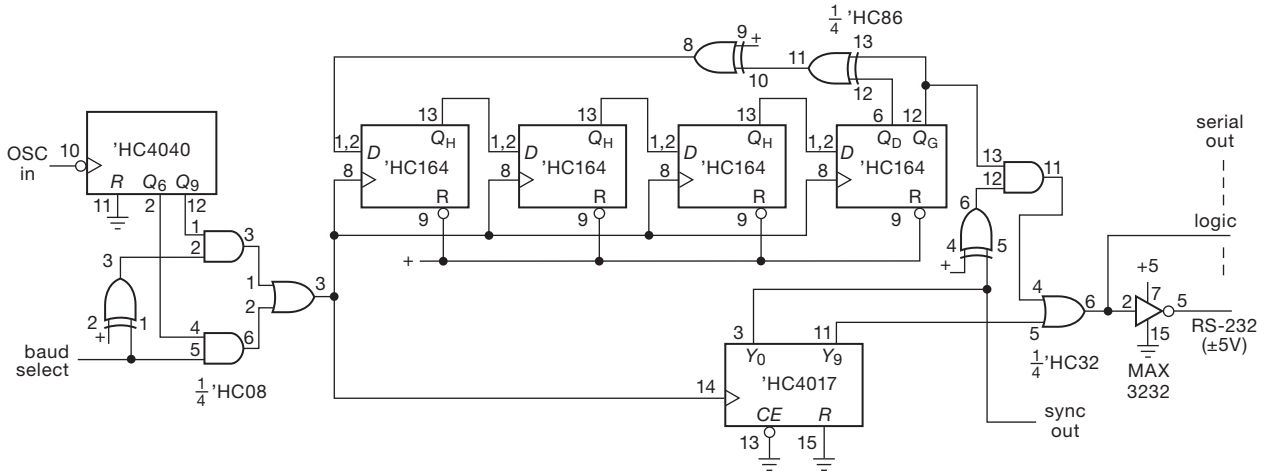


Figure 11.8. Implementation with standard logic (74HC family).

11.3.2 Implementation in standard logic

Gates, counters, shift registers – these are the stuff of "standard logic," most famously the legendary 74xxx logic family, with these discrete logic functions supplied in through-hole (DIP) or the many varieties of surface-mount (SOIC, SSOP, QFN, BGA) packages. In Figure 11.8 we've implemented the random generator in the 74HC subfamily (chosen for wide availability, adequate speed, low power, and DIP/SMT package options). We've indicated DIP pin numbers to emphasize that this is a complete and finished design.

Some comments.

- With standard logic you see tricks like unused XORs converted to inverters, to avoid extra components (such as a package of inverters). Another example is the use of discrete leftover gates (rather than an integrated 2-input MUX) for the baud-rate select.[17]
- We took advantage of some unusual packaged functions, for example the 'HC4017 fully decoded decimal counter, here used to frame the 8 data bits into 10-bit serial characters; the more conventional way would use a separate counter and decode logic. Another helpful packaged function is the 'HC4040 12-stage *ripple* counter (used for the clock subdivision), rather than a cascade of smaller synchronous counters.
- Even with tricks like these, this design required many IC packages (nine, to be exact, not counting the RS-232 driver) – a hallmark of digital implementations done in standard logic. Those nine ICs have to be wired; and, once wired, any changes require at least a wiring change. Furthermore, there are six different IC types, so for this kind of implementation you have to keep a substantial inventory of different IC functions.

[17] Note, however, that you *can* get logic in convenient smaller bites, in the form of small pincount packages with 1–3 gates; these go by names like "TinyLogic™" (Fairchild's name), "PicoGate™" (NXP's name), "MiniGate™" (ON Semiconductor's name), "Little Logic™" (TI's name), Single Gate (ST's name), or LMOS™ (Toshiba's name). We'll call them all "mini-logic," a name evidently overlooked in the trademarking frenzy of the big guys. They come in handy when you need some glue between large digital chips.

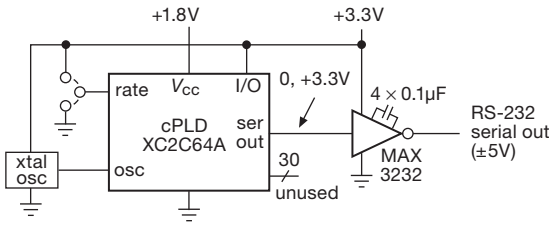


Figure 11.9. Programmable logic permits a far simpler circuit implementation. The connections within the cPLD can be specified, using software tools, either via graphical *schematic entry* or by a text-based HDL; we illustrate both. The resulting desired "fuse" configuration is programmed into the cPLD's internal non-volatile memory, with the cPLD in its actual circuit, via four dedicated pins (known as JTAG, not shown here).

11.3.3 Implementation with programmable logic

As we remarked, programmable logic devices usually provide a better hardware implementation of digital logic, and this example is no exception. Here programmable logic is

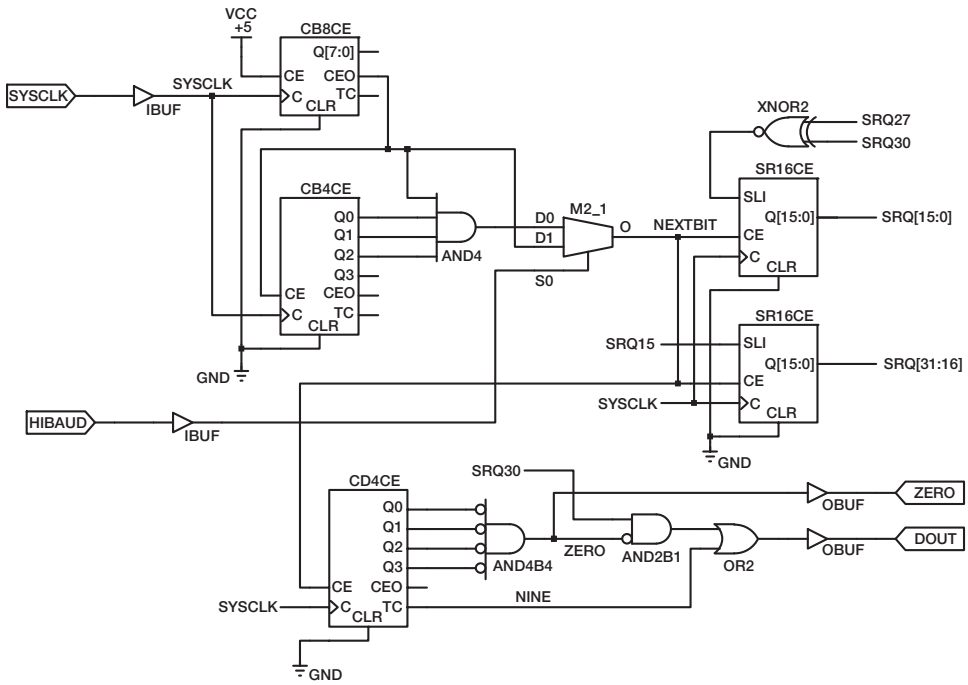


Figure 11.10. Graphical-entry schematic capture for programmable logic design, using Xilinx tools. The component blocks can be generic logic functions (e.g., CB4CE = counter, binary, 4 bits, with chip-enable), or specific standard logic part numbers (e.g., "163" = 74xx163 = 4-bit binary synchronous loadable counter with 3-state outputs and synchronous load and reset; not used in this design); or they can represent modules specified by a text-based hardware description language such as VHDL.

a *much* better choice than standard logic, with a single chip replacing nine; and, as with programmable logic generally, you can reprogram the circuit to fix errors, or to add features.

This design is small, about right for a cPLD, rather than the generally much larger FPGAs. Figure 11.9 shows the wiring, using a smallish cPLD (64 macrocells). The particular cPLD we chose runs its core at 1.8 V, but permits input and output operation from 1.5 V to 3.3 V. This task uses little of the chip's I/O capabilities; 30 pins (of 34 available) are unused. It does, however, use most of the 64 available flip-flops. The chip costs about $2.

A. Programmable logic – schematic entry

Many circuit designers favor a graphical "schematic-entry" technique for designing programmable logic. Figure 11.10 shows how it looks for our pseudorandom byte generator, entered using the tools supplied by Xilinx.

The individual symbols can be familiar gates (OR2: 2-input OR), or lopsided gates (AND2B1: 2-input AND with one bubbled input), or larger generic functions (CD4CE: counter, decimal, 4 bits, with chip enable). However, you can draw a modular block to represent a larger thing, for example a number of interconnected blocks. Here, for example, the entire figure could be represented by a block with two inputs (SYSCLK and HIBAUD) and two outputs (ZERO and DOUT); that block could then be used as a component in a larger system diagram. Furthermore, you can use text-based HDL entry (see below) to define the guts of a block, which you then connect up graphically just like the blocks here. And you can even create the HDL (which defines such a block) with graphical tools. For example, you can implement a state machine by entering the state-transition diagram in a tool like Xilinx's StateCAD (or Altera's Quartus); it generates HDL output (Verilog, VHDL, or ABEL), which can then be represented as a graphical block, with input and output pins.

Graphical-entry software tools let you simulate the design, to make sure it does what you intended; then it performs a "fit" or "place and route" to create the connection pattern for a particular hardware PLD (cPLD or FPGA). Finally, as described earlier, you program the part itself, either in a "device programmer" (from a company such as BP Microsystems) or (more commonly) with the part already

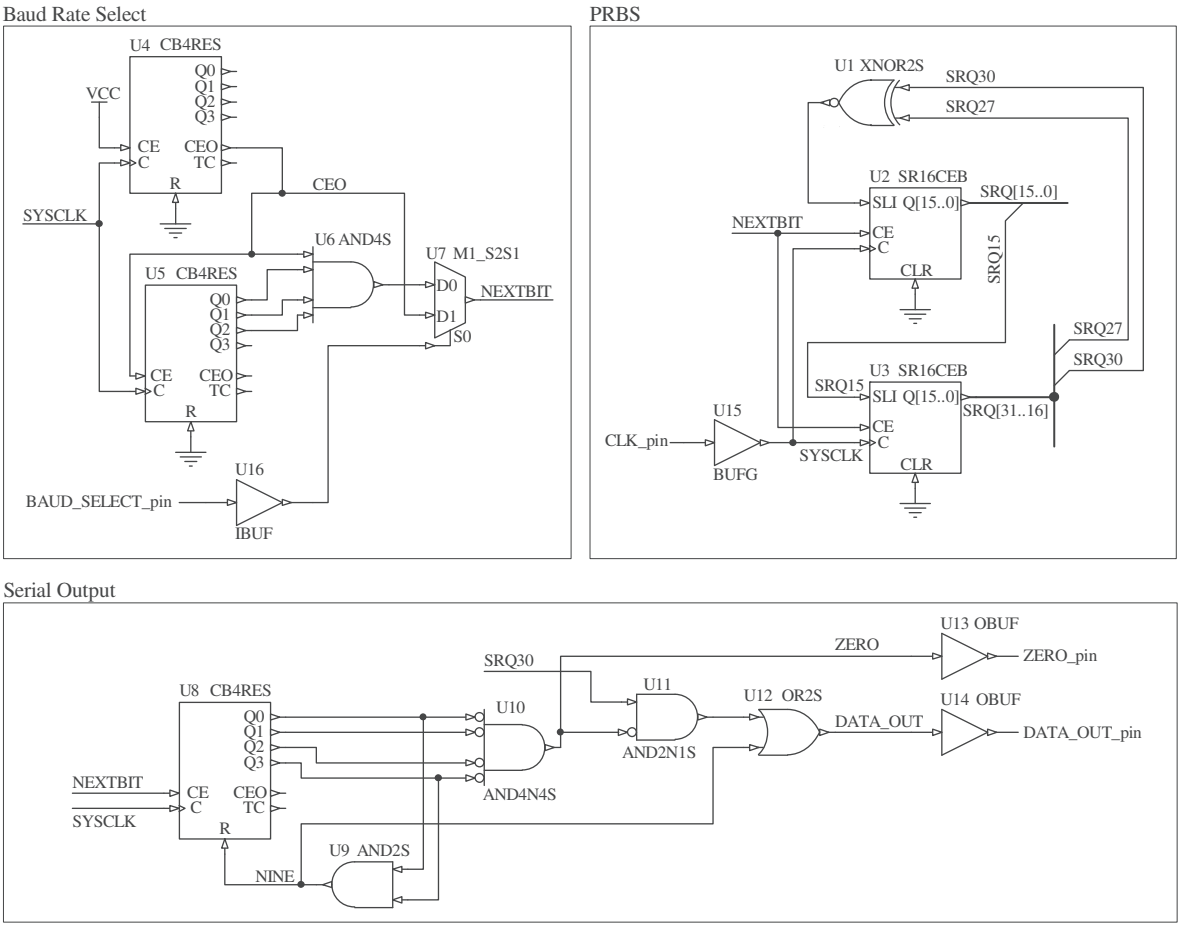


Figure 11.11. The same design as Figure 11.10, implemented with the Altium "Designer" software, and debugged using their NanoBoard development board.

mounted and powered in-circuit. The latter is done through a few dedicated programming pins, which you drive with a programming *pod* that connects to a computer running the design software (typically via USB).

In this example the schematic entry provides a reasonably readable circuit. The designer[18] chose a fully synchronous implementation (all clocked devices are driven from the same clock, namely the input oscillator), explaining that, for some PLDs, shift registers or counters are not guaranteed to work properly when driven from a derived clock (e.g., from the output of a ripple counter), even though this is normal practice when implemented with discrete standard logic.[19]

One traditional complaint with schematic entry has been that the software tools tend to be specific to one vendor's hardware (as in this example). That makes it hard to migrate a design to a different supplier's parts. But recently there have become available some reasonably priced schematic-entry and simulation suites that let you choose among several vendors' FPGAs. We've been trying out such a product – the "NanoBoard" series of hardware development boards supplied by Altium for use with their "Designer" software – that currently support FPGAs from Altera (Cyclone series), Lattice (ECP2 series), and Xilinx (Spartan and Virtex series). If you stick to the generic parts libraries, your design is portable. An example is shown in Figure 11.11, once again our friend the pseudorandom bit

[18] We thank Jim MacArthur for taking the challenge and producing both diagram and working chip in an hour or two.

[19] Evidently the signal delays as such a clock signal is routed in the PLD causes unacceptable skew (§10.8.2C); only the "global clock signals" are guaranteed to work reliably, a fact somewhat buried in the fine print.

generator. The designer[20] chose to group the design into functional modules, to keep the schematic readable. With the exception of the input and output buffers (which were taken from the Xilinx-specific library), all the parts are generic, and so the design is portable with little effort.

At a higher level of abstraction (and cost), you can get synthesis software toolsets like the elegant "Synphony HLS" (High-Level Synthesis) from Synopsys. You start your design with MATLAB and Simulink (from The MathWorks™), which let you snap together Lego-like functional modules from a toolbox of blocks (digital filter, FFT, sampler, etc.). Synphony takes this description and targets (by way of an HDL) either an FPGA or a full ASIC implementation, along with C-language code for functional simulation, test vector generation, and timing. This is far less laborious than traditional HDL design entry. And, because the initial high-level description is verifiable (and its functional blocks are reliably correct), the result is far less prone to errors than a similarly complex chip that is created from an HDL in the conventional manner. Note, however, that the resulting implementation may not be terribly efficient, in terms of use of silicon resources, speed, and power.

Advantages of schematic entry

Those who favor graphical schematic entry generally explain their choice as (a) they come from a schematic background, so it is natural; (b) they find it easier to learn, to understand, and to explain to others; (c) they like having a graphical schematic as documentation; (d) because modules can be HDL-in-disguise, it has all the power of HDL for complex designs where that is needed; and (e) graphical entry is used in graphical programming languages (GPLs) like LabVIEW, MATLAB®, and Simulink®.

11.3.4 Programmable logic – HDL entry

The alternative to graphical design entry is one of the text-based Hardware Description Languages. The current favorites are Verilog and VHDL, with lots of activity also in applying the C programming language to high-level design.

We think it may be helpful to show first how this design looks in the legacy ABEL HDL, whose statements tend to be "close to the hardware": in ABEL you define *pins* (input and outputs) and *nodes* (internal flip-flops or gates), and then you write *equations* that connect up the signals, for example the gates that determine the D input and clock source for a flip-flop. As with schematic entry, you can simulate (with *test vectors*), after which you run the *fitter* program that creates the connection netlist (called a *jedec file*) for your particular target PLD. Finally, you program the part itself, either in-circuit or (more rarely) in a separate stand-alone device programmer.

A. Programmable logic – HDL entry I (ABEL)

Figure 11.12 shows the ABEL source file for creating the pseudorandom byte generator, using a cPLD with 64 macrocells (each of which can implement a complicated logical function of available signals, delivering the result either combinationally or "registered" in a flip-flop).

A few explanatory comments (working from the top down): (a) lines end with a semicolon, and `//` signifies a comment; (b) the designations `com` and `reg_D` (or `reg_T`) following the directive `istype` specify outputs and nodes as combinational or registered (flip-flops); no designation means an input; (c) an array of signals (called a *set* in ABEL) is written as `[sr30..sr0]` (our 31-bit shift register); (d) NOT, AND, OR, and XOR are written as `!, &, #, $`; (e) variable names not declared as pins or nodes (like `baudclk`) are just *definitions*, to simplify later equations (like `[sr30..sr0].clk = baudclk`); thus, for example, `sr`*n* is the name we have chosen for the *n*th shift register bit, and not a reserved name in the ABEL language; (f) the several pins for a named flip-flop are designated by "dot extensions," for example, `sr0.d = !(sr30.q $ sr27.q)` creates the XOR of the *Q*-outputs of shift register bits 30 and 27 and connects it to the D-input of bit 0 (that's the XOR feedback that generates the pseudorandom bits).

It's worth spending a bit of time puzzling through the code to get a feeling for how you do a circuit in a text-based language; we've commented the code liberally to make it (somewhat) understandable. It's not the most efficient implementation (see below for a truly inspired method!), but it works.

B. Programmable logic – HDL entry II (Verilog)

Simple languages like ABEL were OK for simple cPLD designs (we used it for designs with up to 128 macrocells, with some pain), but somewhere between frustrating and hopeless for the really complex designs for which contemporary FPGAs are so well suited. Although both ABEL and CUPL evolved from their roots in PALASM to incorporate higher-level constructions (like "If-Then-Else"), they were finally abandoned in favor of contemporary languages, principally Verilog and VHDL. These have similar capabilities, but, sadly, divide humanity into two camps,

[20] The ever-clever Curtis Mead, to whom we owe thanks.

```
// -------------------------------------------------------
module PRBYTES
title 'pseudo random byte generator - async serial
       Paul Horowitz, 10 Dec 07, rev 5'

// PRBYTES device 'XC2C64A';    xilinx zeropower cPLD

// Inputs
     osc      pin;  // 2.4576 MHz osc input (9600 baud x 256)
     rate     pin;  // LOW selects 1200 baud, HIGH selects 9600 baud

// Outputs
     serout   pin   istype 'reg_D';          // serial data
     syncout  pin   istype 'com';            // RS-232 START pulse (for testing)

// Nodes
     [sbc3..sbc0]   node   istype 'reg_D';   // serial bit counter, BCD (10 states, 0-9)
     [brd10..brd0]  node   istype 'reg_T';   // baud rate divider: divide by 256 or 2048
     [sr30..sr0]   node   istype 'reg_D';    // 31-bit shift register for pseudoran generator

// Constants & Declarations
     high = ^b1;         // high
     low = ^b0;          // low
     baudclk = rate & brd7.q # !rate & brd10.q;        // baud rate clock
     sbczero = !sbc3.q & !sbc2.q & !sbc1.q & !sbc0.q;  // serial bit counter is zero
     sbcnine = sbc3.q & !sbc2.q & !sbc1.q & sbc0.q;    // serial bit counter is nine


Equations
         // baud rate divider: 11 bit binary ripple ctr, use bits 7 or 10 for baudrate
     [brd10..brd0].t = [1,1,1,1,1,1,1,1,1,1,1];  // all flops set to toggle
     brd0.clk = osc;                             // clock lsb from external osc
     [brd10..brd1].clk = [brd9..brd0].q;         // all higher bits clock from previous Q output
                                                 // i.e., brd10.clk = brd9.q; brd9.clk = brd8.q; etc

         // serial bit counter: synch BCD up-counter (4-bit, 10 states: 0-9)
     [sbc3..sbc0].clk = baudclk;   // serial bits clocked at baud rate
     sbc0.d = !sbcnine & !sbc0.q # sbcnine & low;      // lsb toggle, except reset after state 9
     sbc1.d = !sbcnine & (sbc0.q $ sbc1.q);            // toggle if lower bits set
     sbc2.d = !sbcnine & ((sbc1.q & sbc0.q) $ sbc2.q); // useless 'sbcnine & low' term of top line
     sbc3.d = !sbcnine & ((sbc2.q & sbc1.q & sbc0.q) $ sbc3.q);  // was included for clarity only

         // 31-bit xor-feedback shift register to generate pseudo-ran bits
     [sr30..sr0].clk = baudclk;     // prbs shift register clocked at baud rate
     [sr30..sr1].d = [sr29..sr0].q; // compact form for sr30.d = sr29.q; sr29.d = sr28.q; etc
     sr0.d = !(sr30.q $ sr27.q);    // xor feedback to make prbs; negation ensures startup from reset

         // append start and stop bits to generate RS-232 style async serial output bytes; do syncout
     serout.clk = baudclk;  // serial output clocked at baud rate
     serout.d = (!sbczero & sr30.q # sbczero & low) # sbcnine & high;
         // stop bit (high) on nine, start bit (low) on zero, random on 1 through 8
         // RS-232 drivers invert; second term in parenthesis, and 'high', are both unnecessary

     syncout = sbczero;             // RS-232 start pulse

end PRBYTES
// -------------------------------------------------------
```

Figure 11.12. ABEL code for a pseudorandom byte generator.

with designers swearing passionate allegiance to one or the other.

To give a sense of these powerful text-based design languages, we've coded the pseudorandom byte generator design in both Verilog and VHDL.[21] Statements in these languages can be written in terms of the *structure* (e.g., what signals are connected to register inputs), or, at a somewhat higher level, in terms of *behavior* (e.g., what should happen next). These examples are primarily coded behaviorally, which is the mode generally favored by experienced users; but we show also a structural alternative for a small portion (the serial bit counter). Figure 11.13 shows the Verilog design, coded behaviorally; and Figure 11.14 shows alternative Verilog coding for the 4-bit BCD counter portion that is predominately structural.

Some explanatory comments. (a) The Verilog code is recognizably similar to the analogous ABEL code, particularly in the structural form. (b) The *wire* declaration creates an internal node, and *assign* makes a definition. (c) In common with computer programming languages, Verilog and VHDL make plentiful use of conditional structures like `if, else if, else`. (d) Designers tend to favor behavioral statements (being more compact and easily understood), at least through validation by simulation; however, structural coding often produces a more efficient implementation.[22]

A further detail: in this example there are two clocks – the fast ∼2.5 MHz input clock, and a subdivided clock at the baud rate (1.2 kHz or 9.6 kHz, according to the selected baud rate) for the serial bit counter. So the system is not fully synchronous (although each counter is synchronous, by itself). This is OK for a design that will be implemented as a full-custom IC, but it may create difficulties if targeted for a cPLD, or some FPGAs.[23] A clean solution is to use a single fast global clock for both counters, and then to enable the (slower) serial bit counter only at the subdivided rate.

C. Programmable logic – HDL entry III (VHDL)

The *other* popular HDL these days is VHDL, a more strongly typed language with roots in the programming language Ada (Verilog derives more closely from the C programming language). It tends to be more verbose. For the VHDL coding of the pseudorandom byte generator we've taken a fully synchronous approach; that is, the ∼2.5 MHz input clock is used for both the baud-rate divider and the serial bit counter, with the latter enabled only for one clock cycle (when `nextbit` is true). This mirrors the scheme used in the schematic entry examples of Figures 11.10 and 11.11. The (rather lengthy) code is shown in Figure 11.15.

Advantages of HDL entry

Those who favor text-based HDL entry generally explain their choice thus: (a) design entry is rapid, particularly when portions of previous designs are used; (b) the design is more concise (thus easier to know that it is right), and self-documented (as a textual description); (c) the design cycle is streamlined, as a design is being iterated through simulation and prototype; (d) it is particularly easy to change parameters (such as number of bits in a register, ALU, etc) by merely resizing arrays (compared with rearranging graphical modules); (e) the languages are standardized and universal (compared with proprietary schematic-entry tools); (f) good simulators are available and free of charge; (g) HDL languages are best suited for high-level synthesis of complex designs (like a microprocessor), and prefab open-source IP cores are provided as HDL;[24] (h) HDL entry is well suited to implementation as a *full-custom* IC (ASIC), which is often desirable for a circuit design destined for large-quantity manufacture and a high degree of optimization in speed and power (but which start out as an FPGA during development and small-quantity production); and (i) for those with a programming background, HDL entry is more natural.

11.3.5 Implementation with a microcontroller

Micro*controllers* (the subject of Chapter 15) are inexpensive processors intended for use in things *other than* computers. You should think of them more as a circuit component and less as a computer. Compared with a micro*processor* intended for a computer, microcontrollers are intended to stand alone, and therefore always include on-chip program and data memory, along with a selection of "peripherals" such as communications (USB, firewire, Ethernet, UART, CAN, SPI, I^2C), ADCs, digital I/O, comparators, LCD display drivers, pulse-width modulators, timers, and the like. Many also include an internal oscillator (all they need is dc power!), in some cases accurate enough for

[21] We are indebted to our colleagues GuYeon Wei and Curtis Mead, who kindly coded this example.

[22] An example might be a 32-bit counter, where a behavioral statement like "`count=count+1`" might compile to a register and 32-bit full adder, the latter receiving a constant 00000001(hex) as one of its inputs.

[23] Some FPGAs let you drive internal clock distribution lines from logic outputs, but some don't, for example the Virtex-5 and Spartan-3, respectively; for the latter you would have to bring the clocking signal out to a pad.

[24] But IP cores that must be licensed are encrypted.

```
// ------------------------------------------------------------------------------------
// Pseudo random byte generator - async serial
// Gu-Yeon Wei and Curtis Mead; "behavioral" coding

module PRBYTES(osc, rate, reset, serout);

// inputs
// osc: 2.4576 MHz oscillator input (9600 baud x 256)
// rate: 0 = 1200 baud, 1 = 9600 baud
// reset: active high reset to clear out counters
input osc, rate, reset;

// outputs
// serout: serial data
output serout;

// define registers
reg serout;      // a "reg" is a node that holds its value until overwritten
reg [3:0] sbc;   // RS-232 serial bit counter
reg [10:0] brd;  // baud rate divider
reg [30:0] sr;   // PRBS shift register

// define wires
wire baudclk, sbczero, sbcnine; // a "wire" is a combinational circuit node

// logic
assign baudclk = rate & brd[7] | ~rate & brd[10];        // baud rate clk, div 256 or 2k
assign sbczero = ~sbc[3] & ~sbc[2] & ~sbc[1] & ~sbc[0];  // serial bit counter zero flag
assign sbcnine = sbc[3] & ~sbc[2] & ~sbc[1] & sbc[0];    // serial bit counter nine flag

// baud rate divider: 11 bit binary ctr, use bits 7 or 10 for baudrate
always @(posedge osc)  // logic clocked by osc
 begin
   if (reset)
      brd <= 0;
   else
      brd <= brd+1;
 end

// serial bit counter: synch BCD up-counter (4-bit, 10 states:0-9)
// this is "behavioral"; see later program fragment for "structural"
always @(posedge baudclk)  // logic clocked by baudclk
 begin
   if (reset)
      sbc <= 0;
   else if (sbcnine)
      sbc <= 4'b0000;
   else
      sbc <= sbc + 1;
 end

// 31-bit xor-feedback shift register to generate pseudo-random bits
always @(posedge baudclk)
 begin
   if (reset)
      sr[0] <= 0;  // prevent stuck state of all ones
   else begin
      sr[30:1] <= sr[29:0];
      sr[0] <= ~(sr[30] ^ sr[27]);  // xnor makes all ones the stuck state
  end
 end

// serial output stream comes from last SR bit, but overridden by "0" start, and "1" stop
always @(posedge baudclk) begin
   serout <= (~sbczero & sr[30]) | sbcnine;
end

endmodule //PRBYTES
// ------------------------------------------------------------------------------------
```

Figure 11.13. Behaviorally-coded Verilog for pseudorandom byte generator.

```
// ---------------------------------------------------------------------
// alternative structural coding of serial bit counter:
// synch BCD up-counter (4-bit, 10 stages:0-9)
always @(posedge baudclk) begin
   sbc[0] <= ~sbcnine & ~sbc[0] & ~reset;
   sbc[1] <= ~sbcnine & (sbc[0] ^ sbc[1]) & ~reset;
   sbc[2] <= ~sbcnine & ((sbc[0] & sbc[1]) ^ sbc[2]) & ~reset;
   sbc[3] <= ~sbcnine & ((sbc[0] & sbc[1] & sbc[2]) ^ sbc[3]) & ~reset;
end
// ---------------------------------------------------------------------
```

Figure 11.14. Structurally-coded Verilog fragment for 4-bit BCD counter.

timing of on-chip serial ports (e.g., ±2%). They are programmed (and reprogrammed) in-circuit, by a serial connection like the JTAG boundary-scan protocol.[25] Recent designs include internal circuitry that permits run-time debugging in-circuit (by the same JTAG port used for programming).

Microcontrollers (sometimes abbreviated μC) are inexpensive (typically ranging from less than a half dollar up to ten dollars), and are available in literally thousands of versions from dozens of manufacturers; popular offerings are the PIC series (from Microchip), the AVR series (from Atmel), the ARM series (from multiple vendors), and evolved incarnations of the legacy Intel 8051 series (multiple vendors). Low-end μCs are usually 8-bit processors, with perhaps 1 kB of program memory and 128 bytes of RAM, and clock speeds of 20 MHz; at the high end there are 32-bit processors with 512 kB program memory, 64 kB of RAM, and many integrated peripherals.[26]

Microcontrollers are wonderfully versatile and capable, and they are found in nearly everything electronic, from toasters and toothbrushes to trucks and traffic lights. They can do a pseudorandom byte generator with the greatest of ease. Figure 11.16 shows the "circuit" – basically just dc power to the chip, and bytes out! The rest of the task is the programming, which we'll illustrate two ways.

A. Microcontroller programming – assembly code

Microcontrollers, and indeed all processors, perform a sequence of operations, specified as instructions that are stored in some form of electronic memory (see Chapter 14); in the case of μCs the program is held on-chip in non-volatile memory (thus retained when power is off). The instruction set is particular to the processor type, and includes operations such as arithmetic (e.g. `add`), logical (e.g. `shift`), data transfer (e.g. `move`), and branching (e.g. `brz`, branch if zero). The instructions themselves reside in memory as a set of bytes, fetched by the processor during program execution.

As with computer programming generally, programmers rarely deal with this *object code* itself (which is both tedious and error-prone), programming and debugging instead at a more humanly understandable level: either in *assembly language*, or in a higher-level language (usually C or C++). The code so written is then *assembled* or *compiled* (using software) to produce the μC-readable binary object code that will reside in the processor's program memory. There are nice software tools to help you get the code right – for example, *simulators* that let you see how the candidate program runs, step-by-step; and *run-time debuggers* that let you watch what the μC is actually doing, in-circuit, when it runs your program. The term *Integrated Development Environment* ("IDE") is used to describe a software suite that includes compiler, assembler, loader, and run-time debugger. You want one of these.

Figure 11.17 is a listing of assembly code for the pseudorandom byte generator implemented in the microcontroller circuit of Figure 11.16; not to worry, we'll see it coded in C, finally, in Figure 11.18.

Just a few notes of explanation (hey, this isn't a book on software!). (a) All microcontrollers include a bunch of internal "special-function registers," whose contents you must initialize to set up timer modes, interrupts, location of initial stack pointer, and so on. That's what the first seven `mov` operations do, here. This can be fussy code, particularly involving timer modes and baud rates (best to copy it from a program known to work correctly). And,

[25] A 4-wire serial protocol, which can be chained through a succession of ICs, and which permits run-time loading of, examination of, and bypassing of some internal registers in complex ICs; see §14.7.4.

[26] A current example is the ARM7 LPC2458 from NXP (formerly Philips), displayed in Figure 10.86: 512kB/64kB memories, 72 MHz clock, 10/100 Ethernet, USB 2.0, 10-bit A/D and D/A, 2xPWM, 4 UARTs, 2xCAN, SPI, 2xSSP, 3xI^2C, I^2S, 136 bits of general-purpose I/O, and an external memory controller. This puppy costs just $10! For higher clock speeds you would choose instead the faster ARM9.

```
--------------------------------------------------------------------------------
-- Pseudo random byte generator - async serial
-- Curtis Mead, modeled after Gu-Yeon Wei's Verilog
-- but implemented as a fully synchronous design
-- COMMENTS begin with two dashes (--)

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PRBS is
    Port ( CLK          : in   STD_LOGIC;  -- 2.4576 MHz oscillator
           RESET        : in   STD_LOGIC;  -- Synchronous reset, active high
           BAUD_SELECT  : in   STD_LOGIC;  -- 0 = 1200 baud, 1 = 9600 baud
           ZERO_pin     : out  STD_LOGIC;  --
           DATA_OUT_pin : out  STD_LOGIC); -- Serial data output
end PRBS;

architecture Behavioral of PRBS is
    signal sbc         : STD_LOGIC_VECTOR( 3 downto 0); -- serial bit counter
    signal brd         : STD_LOGIC_VECTOR(10 downto 0); -- baud rate divider
    signal prbs_bits   : STD_LOGIC_VECTOR(30 downto 0); -- prbs shift register
    signal baudtemp    : STD_LOGIC_VECTOR( 1 downto 0); -- brd temp register
    signal nextbit     : STD_LOGIC; -- pseudo clock signal for serial bit counter
    signal sbcnine, sbczero     : STD_LOGIC;

begin

-- PRBS with 32 bit shift register, bits 27 and 30 XNOR'ed into input
process (CLK)
begin
   if CLK'event and CLK='1' then
      if RESET ='1' then
         prbs_bits(0) <= '0';
      elsif nextbit = '1' then
         prbs_bits(30 downto 1) <= prbs_bits(29 downto 0);
         prbs_bits(0) <= prbs_bits(27) xnor prbs_bits(30);
      end if;
   end if;
end process;

-- baud rate divider, 11 bit counter, synchronous reset
-- use bit 7 for 9600 baud and bit 10 for 1200 baud
process (CLK)
begin
   if CLK='1' and CLK'event then
      if RESET='1' then
         brd <= (others => '0');
      else
         brd <= brd + 1;
      end if;
   end if;
end process;
```

Figure 11.15. Fully synchronous VHDL code for pseudorandom byte generator (continued below).

```
-- shift register used to set nextbit for one clock only,
--  triggered by baudrate divider bits
--  for clocking nextbit from baud rate divider output 'baudtemp'
process (CLK)
begin
   if CLK='1' and CLK'event then
      if RESET='1' then
         baudtemp <= "00";
      else
         baudtemp(1) <= baudtemp(0);
         baudtemp(0) <= (BAUD_SELECT and brd(7)) or (not BAUD_SELECT and brd(10));
      end if;
   end if;
end process;
nextbit <= baudtemp(0) and not baudtemp(1); -- nextbit high for one clock cycle

-- serial bit counter, 4-bit BCD up with synchronous clear
process (CLK)
begin
   if CLK='1' and CLK'event then
      if RESET = '1' then
         sbc <= (others => '0');
      elsif nextbit='1' then
         if sbcnine='1' then
            sbc <= (others => '0');
         else
            sbc <= sbc + 1;
         end if;
      end if;
   end if;
end process;
sbczero <= '1' when sbc = "0000" else '0'; -- serial bit counter zero flag
sbcnine <= sbc(0) and sbc(3); -- serial bit counter nine flag

-- outputs synchronous with clock signal CLK
process (CLK)
begin
    if CLK='1' and CLK'event then
        if RESET = '1' then
            DATA_OUT_pin <= '0';
            ZERO_pin <= '0';
        elsif nextbit = '1' then  -- nextbit is an "enable" (at the baudrate)
            DATA_OUT_pin <= (not sbczero and prbs_bits(30)) or sbcnine;
            ZERO_pin <= sbczero;
        end if;
    end if;
end process;

end Behavioral;
--------------------------------------------------------------------------------
```

Figure 11.15 continued.

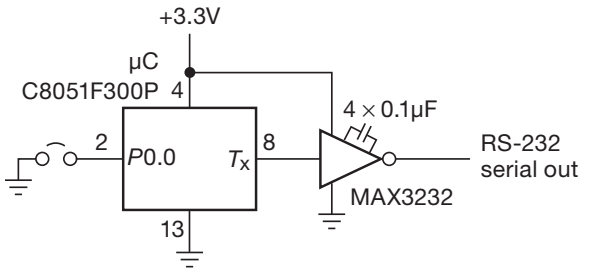


Figure 11.16. Microcontrollers represent a compact alternative to programmable logic, particularly when speed is not critical. This particular microcontroller choice (among literally thousands) includes an internal oscillator, so *no* external components are needed to generate the logic-level serial bitstream output.

surprise, there's absolutely no standardization across different microcontroller families. Fussy, fussy, fussy… (b) The program bookends `org 00h` and `end` tell the assembler where to put the first instruction, and where the code ends. That is, they are not instructions that the microcontroller executes, but rather directives to the assembler; they're sometimes called "pseudo-ops." (c) The rest of the code consists of arithmetic and logical operations, such as clear, copy (called `mov`), compare (`cjne`: compare and jump on not equal), loop counters (`djnz`: decrement and jump on nonzero), subroutine call (`acall`), and so on.

You really don't want to fall in love with this stuff – we show it mostly to warn you off!

B. Microcontroller programming – coding in C

Assembly language programming is fussy stuff, and it's easy to make trivial mistakes. Even when you get it right, it's tricky to modify. Large programs are laborious. The code must be rewritten if you want to use it with a different family of microcontroller. And, worst of all, you probably won't understand what you've written a few weeks later, when it needs to be changed.

For these reasons most programmers prefer a high-level language like C or C++ for coding.[27] It's easier to write and understand, and it's portable across microcontroller families (though with some changes required to accommodate architectural differences). Figure 11.18 list the PRBS generator, in the C programming language.

The programmer[28] noticed, in a burst of inspiration, that the PRBS could be coded to perform a full byte at a time, rather than the ponderous bit-at-a-time coding we assumed necessary in the earlier examples. That's because the shift register has, at any moment, all the downstream bits you need to calculate all the fresh bits up to the first feedback tap (in this case 27 bits). Given the byte organization of 8-bit μCs and the fact that the built-in serial-port UART expects a full 8-bit quantity before transmission, the smart thing is to create a new upstream byte, using the bytewise XOR instruction (bitwise XOR, without carry). The resulting program runs wicked fast! See if you can figure out the coding.

11.4 Advice

So, what should you do? Here's a brief summary of recommendations, which we think fairly represents the ups and downs of the various choices you have for digital (and mixed-signal) implementations.[29] We'll do this two ways, first by *Technology*, then by *User Community*.

11.4.1 By *Technologies*

- **Standard logic**
  - `Upside`: OK if you want to put a simple circuit together quickly and you have the parts on hand; useful for bus drivers, buffers, and small logic "glue" (for sticking large chips together, or for fixing logic errors); available in through-hole, so you can use a pluggable breadboard, or a solder (or wirewrap) protoboard; no software design tools needed.
  - `Downside`: unwieldy for complex circuits; multiple functions to inventory; inflexible.
- **Programmable logic (cPLD, FPGA)**
  - `Upside`: generally preferred (over standard logic) for most digital designs – reduced package count, easily reprogrammed, flexible and inexpensive, small inventory needed; favor cPLDs for predictable timing and for small designs, FPGAs for large designs (though small FPGAs are emerging); best choice for fast parallel logic and state machines; good for timing-critical functions (such as communications protocols), bit-level operations (CRC, Viterbi), and functions with lots of I/O; designs can be migrated across PLD families, and to full-custom ICs.

[27] Perhaps with small blocks of assembly code inserted where speed is critical, for example an interrupt service routine (see Chapter 13), within a larger program.

[28] Jason Gallicchio, to whom we are indebted for this code and for other assistance with the microcontroller discussion.

[29] Note, however, that it's usually smoother sailing if you adopt the methods that people around you are using – their experience base and toolsets accelerate learning and debugging – even if some different technique potentially offers other advantages.

```
// ------------------------------------------------------
;; program to send pseudo-random bytes from serial UART  rev of 12/14/07

          org 00h;          starting address at power up or reset

          mov IE, #00h;     disable interrupts
          mov SP, #90h;     init stack pointer, used for subroutine
          mov PCON, #00h;
          mov TMOD, #20h;   timer 1: 8 bit, auto reload
          mov TCON, #40h;   timer 0 off, timer 1 on
          mov SCON, #40h;   8 bit UART, tx only, set by timer 1
          mov TH1, #0FDh;   9600 baud (reload value = 253 decimal)

          clr A;
          clr TI;
          jb P1.0, makebyte; logic HIGH input on P1.0 pin = 9600 baud
          mov TH1, #0E8h;   LOW: 1200 baud (reload value = 232 decimal)

makebyte: mov R3, #8;       setup loop counter - 8 shifts make a byte

             ; create feedback bit as XNOR of bits 27 and 30 (0..30)
makebit:  mov C, ACC.4;     that is, ACC.4 and ACC.1
          mov R2, #0;
          mov R2.1, C;      stick ACC.4 into R2.1, to XNOR with masked ACC
          anl A, #2;        mask ACC, preserving ACC.1 only
          clr C;
          cjne A, R2, loadreg;  XNOR=0, carry already cleared
          setb C;           XNOR=1, set carry

loadreg:  mov R0, #80h;     80h-83h will hold MSByte-LSByte of 32-bit SR

shift32:  mov A, @R0;       RO points to one of 4 bytes at 80h..83h
          rrc A;            shift right through carry
          mov @R0, A;       stash it
          inc R0;
          cjne R0, #84h, shift32;  check for last byte of 4

          djnz R3, makebit; do it 8 times
          acall sendbyte;   send finished byte via serial UART
          sjmp makebyte;    and start next byte

             ; code for sending one byte
sendbyte: jnb TI, sendbyte; wait for last tx done
          mov SBUF, A;      load transmit buffer
          ret;

          end
// ------------------------------------------------------
```

Figure 11.17. Assembly code for pseudorandom byte generator.

```
// ---------------------------------------------------------
#include <avr/io.h>

#define F_CPU  20000000  // 20 MHz
#define BAUD_RATE  9600      // or 115200

void USART_Init(void) {
  UBRR0 = F_CPU/BAUD_RATE/8 - 1; // Set baud rate
  UCSR0A = (1<<U2X0);  // Change baud divisor from 16 to 8
  UCSR0B = (1<<RXEN0)  | (1<<TXEN0);   // enable Rx & Tx
  UCSR0C = (1<<UCSZ01) | (1<<UCSZ00); // 8N1, no parity
}

void putch(unsigned char ch)
{
  while ( !(UCSR0A & (1<<UDRE0)) );
  UDR0 = ch;
}


int main(void)
{
  unsigned char out;
  unsigned char d=0xff, c=0xff, b=0xff, a=0xff;

  USART_Init();

  while(1) {
    out =  ( (d<<1) | (c>>7) ) ^ c;  // 31 bit, tap 24
    d = c;
    c = b;
    b = a;
    a = out;

    putch(out);
  }
}
// ---------------------------------------------------------
```

Figure 11.18. C code for pseudorandom byte generator.

- – `Downside`: time to learn software tools (often proprietary), and their cost; surface-mount parts only (requires custom PCB or adapter); cost of programming pod and design tools; short life cycle (supply problems).
- – `PLD design--program tools`
  - ∗ *Schematic entry*
    - · `Upside`: easy to learn, understand, and explain to others (for circuit nerds especially); self-documenting; entry of parallel circuitry in a parallel way; analogous graphical programming languages (GPLs) used in tools like LabVIEW, MATLAB®, and Simulink®.
    - · `Downside`: proprietary design tools; cost; limited applicability.
  - ∗ *HDL (Verilog/VHDL)*
    - · `Upside`: natural migration for programmer nerds who can adapt to parallel thinking; concise and standardized; cost-free simulation and compilation tools; easy-to-iterate design modifications; can migrate across PLD families, and to full-custom ICs;

well suited for large complex designs; IP core availability.
  - `Downside`: time to learn, and lack of schematic documentation (especially for non-programmers).
- **Microcontrollers** (see Chapter 15!)
  - `Upside`: programmable, flexible embedded computer; ideal for complex state machines with lots of decisional branching; rapid iteration of code and (in-circuit) debugging; many on-chip peripherals (communication,[30] conversion, interface,[31] other[32]); familiar programming language (C, C++); best for embedded control, especially involving user communication (display and control); enjoyable!
  - `Downside`: slower than cPLD/FPGA; less suited to bit-level manipulation; less parallelism.
  - μ`C program tools`
    - *Assembly language*
      - `Upside`: can handcraft optimized code, for example an interrupt service routine.
      - `Downside`: fussy and error prone; difficult to make major modifications; cannot port to a different processor family.
    - *C, C++*
      - `Upside`: standardized and portable (though some processor-specific changes needed); widespread programming expertise; structured language facilitates changes and upgrades; libraries of useful functions; well suited for complex tasks.
      - `Downside`: not much... compiled code may be less efficient in size and speed (can embed assembly code for critical loops and routines, and for access to special features); code can be hard to understand; requires compiler toolset (some are expensive, others are free).
    - *Other (Basic, Arduino Software; Java, Python)*
      - `Upside`: easy to understand, good for entry level.
      - `Downside`: limited range of microcontroller support, not generally portable across processor families, not for big projects.

11.4.2 By *User Communities*

All of the user communities are well served by *microcontrollers*, owing to low cost, ease of programming, and flexibility. Among these communities, the following additional technologies are favored.

- **High-volume production**[33]
  Here unit cost and time-to-market are critical. In large volumes, unit cost is minimized with ASICs (unique Application Specific full-custom Integrated Circuits) and ASSPs (generally available Application-Specific Standard Products), whereas time-to-market is generally shortened by using FPGAs to eliminate the ASIC design–debug cycle. In large production it is common to use high-density BGA (ball-grid array) packages and other fine-pitch surface-mount (SMT) devices, along with the necessary multilayer circuit boards and advanced assembly techniques. And, of course, always use microcontrollers.
- **Complex prototype and small production**[34]
  Here you usually cannot afford the delays and costs associated with both ASICs and high-density BGA packages. So the favored technologies are cPLDs and FPGAs, along with assorted support chips, all in surface-mount packages, and assembled on multilayer boards.[35] And, of course, always use microcontrollers.
- **Laboratory and "1-Off"**[36]
  Here it's important to be able to put together an instrument relatively quickly, and to modify and evolve its circuitry as the need arises. We favor through-hole prototyping boards, populated with through-hole components, and (where essential) with a small number of SMT devices (on through-hole adapters[37]). This constrains you to standard logic and interface, a limited selection of microcontrollers, and perhaps some cPLDs/FPGAs, along with analog components (op-amps, etc). You can make greater use of FPGAs if you take advantage of third-party pre-fabricated daughterboards from companies like Digilent, DLP Design, or Opal Kelly: these range from bare-bones boards (FPGA, program ROM, and USB or JTAG interface) up to complex boards with displays, Ethernet, and so on.[38]

[30] For example, UART, SPI, I^2C, CAN, USB, Ethernet, IrDA, WiFi, digital audio/video, Bluetooth, ZigBee.

[31] For example, keyboard, mouse, PCI, PCIe, PWM, SIM and Smart Card, GPS, PCMCIA/CF.

[32] For example, external DRAM/SDRAM, MMU, GPS, CCD/CMOS cameras, LCDs, graphic display.

[33] Cellphones, TV set-top boxes, etc.

[34] Oceanographic and scientific research instruments, MRI machines, experimental products, etc.

[35] If you need to use BGA or other high-pin-count parts, it's good to know about prototype assembly services such as Advanced Assembly™: they can handle fine-pitch BGA assembly (including X-ray inspection), and they will even take care of ordering the parts for you. Best of all, they don't mind orders of just a few boards.

[36] The *first* NMR, MRI, STM; atom traps, laser cooling, an astronomical telescope control, etc.

[37] For example from Aries Electronics, or Bellin Dynamic Systems.

[38] If you want to use FPGAs as building blocks in diverse projects, and

Alternatively, you can start with a fully populated microcontroller kit, including PLDs, that may include also unpopulated "prototyping" pads for a small number of additional devices.

- **Hobbyist**[39]
  Here the idea is to have lots of inexpensive fun. The basic unit is the microcontroller, either as part of a prefabricated kit,[40] or used on a through-hole protoboard, or perhaps a custom printed-circuit board.[41] Through-hole components (see *Laboratory and "1-Off", above*) are always easy to use, as are PLDs included in microcontroller kits.

you can't find what you want from companies like these, you can follow the lead of one of our colleagues: he designed a small daughterboard to hold his chosen FPGA, along with a couple of EEPROMs, an oscillator, a JTAG header, and voltage regulators (so it runs from +5 V power). It has a pair of 2×20-pin headers (connected to lots of FPGA pins) aimed downward, to plug into a custom board.

[39] Robots, remote control, lighting systems, audio/radio/video, etc.

[40] Supplied by the μC manufacturer, or from an independent supplier like Parallax, Digilent, DLP Design, or the Arduino project. It's pleasant, also, to loiter at websites like sparkfun.com and adafruit.com.

[41] There are freeware versions of layout tools, such as Eagle (from CadSoft); there are open-source tools; and there are inexpensive PCB fabrication houses, for example Advanced Circuits (www.4pcb.com), who also offer their "PCB Artist" free PCB layout software (which, however, produces files in their proprietary format).

Review of Chapter 11

An A-to-G summary of what we have learned in Chapter 11. This summary reviews basic principles, facts, and application advice in Chapter 11.

¶A. Programmable Logic Devices (PLDs).
PLDs are digital integrated circuits, typically containing thousands to millions of gates (and, sometimes, additional functional blocks such as transceivers, RAM, DSP, or general-purpose microcontrollers), whose circuit interconnections are determined by user programming (§11.2). That is, in analogy with a computer program (which tells the processor *what to do*), the program for a PLD tells the chip *how to connect* its internal component pieces. Some PLDs include non-volatile (and reprogrammable) memory that holds the programmed configuration; others read their configuration from an external non-volatile ROM at power-up, and hold it in on-chip (and volatile) memory; and many allow programming via a serial configuration port (commonly a JTAG port).

¶B. PLD Applications.
Small PLDs (tens to hundreds of macrocells, thousands of gates) are commonly used as "glue logic," replacing discrete gates, flip-flops, registers, and the like. Larger PLDs, and particularly FPGAs (field-programmable gate arrays), can implement nearly a complete system-on-a-chip. The largest FPGAs contain millions of flip-flops, megabytes of RAM, hundreds of I/O ports (LVDS, PCIe, etc.), and dozens of serial transceivers – resources adequate to hold a "soft" processor (i.e., configured from gates and flip-flops) running embedded Linux. (And when thinking of PLDs for some application, be sure to consider the alternative of a general-purpose *microcontroller* – they're increasingly capable, inexpensive, and easy to program.)

¶C. Programming Overview.
PLD design (§11.2.6) starts with the functional specification, where the choice is either *schematic entry* or a *hardware description language* (HDL). Next you run a *simulation*, to verify that the design works as intended. After that a *synthesis* process converts the design to a *netlist* (a full set of logical connections). The netlist is then "fitted" to the target PLD, a process known as *place and route*. Finally, the fitted design is downloaded to the PLD's memory – either to the PLD itself (if it has on-chip non-volatile memory), or to an external configuration ROM (from which the PLD loads at power-up), or possibly hot-loaded from an external processor directly to the powered-up PLD.

PLD design tools are provided by the PLD manufacturers, and by some third parties. For low-end PLDs the tools may be free, but they can be quite expensive for the high-end parts. Considerable time may be needed to master the PLD design tools (which have a distressing tendency to change with time), and the software can be (and usually is) buggy.

¶D. Schematic Entry.
Most comfortable to non-coders is schematic entry, in which you wire up familiar logic symbols (e.g., Figure 11.10) in the same manner as you would do when entering a circuit for a PCB layout. There's additional flexibility, because you can define larger functional blocks that are specified with either schematic entry or HDL text entry.

Another graphical entry method, at a higher level of abstraction, exploits software synthesis tools that convert a MATLAB/Simulink Lego-like functional-block description to an HDL source-code output; the latter can target a PLD or a full-custom (ASIC) implementation, as well as providing simulation, timing, and test vectors.

Those who favor graphical schematic entry generally explain their preference as (a) coming from a schematic background, they find it natural; (b) they find it easier to learn, to understand, and to explain to others; (c) they like having a graphical schematic as documentation; (d) because modules can be HDL-in-disguise, it has all the power of HDL, for complex designs where that is needed; and (e) graphical entry is used in graphical programming languages (GPLs) like LabVIEW, MATLAB®, and Simulink®.

¶E. HDL Entry.
Hardware description languages like Verilog and VHDL dominate the PLD-entry scene. Statements in either language can be *structural* (e.g., what signals are connected from some output to another input) or *behavioral* (e.g., which state follows another, and under which conditions). See the illustrative examples in §§11.3.4B and 11.3.4C.

Those who favor text-based HDL entry generally explain their preference as (a) coming from a programming background, they find it natural; (b) design entry is rapid, particularly when re-using portions of previous designs; (c) the design is more concise (thus easier to know that it is right), and self-documented (as a textual description); (d) the design cycle is streamlined, as a design is being iterated through simulation and prototype; (e) it is particularly easy to change parameters (such as number of bits in a register, ALU, etc) by merely resizing arrays (compared with rearranging graphical modules); (f) the

languages are standardized and universal (compared with proprietary schematic entry tools); (g) good simulators are available, and free of charge; (h) HDL languages are best suited for high-level synthesis of complex designs (like a microprocessor), and prefab open-source IP cores are provided as HDL; and (i) HDL entry is well-suited to subsequent implementation as a *full-custom* IC (ASIC), following a prototype stage implemented initially as an FPGA.

¶F. Advice, by *Technology*.
(Paraphrased shamelessly from §11.4.1.)

- **Standard logic**
  - `Upside`: OK for quick assembly of simple circuits, and useful for bus drivers, buffers, and small logic "glue"; available in through-hole (for quick prototyping); no software design tools needed.
  - `Downside`: unwieldy for complex circuits; multiple functions to inventory; inflexible.
- **Programmable logic (cPLD, FPGA)**
  - `Upside`: better for most digital designs – reduced package count, easily reprogrammed, flexible and inexpensive, small inventory needed; favor cPLDs for predictable timing and for small designs, FPGAs for large designs; best choice for fast parallel logic and state machines; good for timing-critical functions, and functions with lots of I/O; designs can be migrated across PLD families, and to full-custom ICs.
  - `Downside`: time to learn software tools (often proprietary), and their cost; surface mount parts only; cost of programming pod and design tools; short life cycle (supply problems).
  - `PLD design/program tools`
    - *Schematic entry*
      - `Upside`: easy to learn, understand, and explain to others; self-documenting; entry of parallel circuitry in a parallel way; analogous graphical programming languages (GPLs) used in tools like LabVIEW, MATLAB®, and Simulink®.
      - `Downside`: proprietary design tools – cost; limited applicability.
    - *HDL (Verilog/VHDL)*
      - `Upside`: natural migration for programmers; concise and standardized; cost-free simulation and compilation tools; easy to iterate design modifications; can migrate across PLD families, and to full-custom ICs; well-suited for large complex designs; IP core availability.
      - `Downside`: time to learn, and lack of schematic documentation.
- **Microcontrollers** (see Chapter 15!)
  - `Upside`: programmable, flexible embedded computer; ideal for complex state machines with lots of decisional branching; rapid iteration of code and (in-circuit) debugging; many on-chip peripherals (communication, conversion, interface, other); familiar programming language (C, C++); best for embedded control, especially involving user communication (display and control); enjoyable!
  - `Downside`: slower than cPLD/FPGA; less suited to bit-level manipulation; less parallelism.
  - μ`C program tools`
    - *Assembly language*
      - `Upside`: can handcraft optimized code, for example an interrupt service routine.
      - `Downside`: fussy and error prone; difficult to make major modifications; cannot port to a different processor family.
    - *C, C++*
      - `Upside`: standardized and largely portable; widespread programming expertise; structured language facilitates changes and upgrades; libraries of useful functions; well suited for complex tasks.
      - `Downside`: not much… compiled code may be less efficient in size and speed (can embed assembly code where needed); code can be hard to understand; requires compiler toolset (some are expensive, others are free).
    - *Other (Basic, Arduino Software; Java, Python)*
      - `Upside`: easy to understand, good for entry level.
      - `Downside`: limited range of microcontroller support, not generally portable across processor families, not for big projects.

¶G. Advice, by *User Community*.
(Paraphrased shamelessly from §11.4.2.) *All* of the user communities are well served by *microcontrollers*, owing to low cost, ease of programming, and flexibility. Among these communities, the following additional technologies are favored.

- **High-volume production**
  Here unit cost and time-to-market are critical. In large volumes, unit cost is minimized with *ASICs* (unique Application Specific full-custom Integrated Circuits) and *ASSPs* (generally available Application-Specific Standard Products), whereas time-to-market is generally shortened by using *FPGAs* to eliminate the ASIC design–debug cycle. In large production it is common to use

high-density *BGA* (ball-grid array) packages and other fine-pitch surface-mount (SMT) devices, along with the necessary multilayer circuit boards and advanced assembly techniques. And, of course, always use *microcontrollers*.

- **Complex prototype and small production**
  Because of the delays and costs associated with both ASICs and high-density BGA packages, the favored technologies are *cPLDs* and *FPGAs*, along with assorted support chips, all in surface-mount packages, and assembled on multilayer boards. And, of course, always use *microcontrollers*.
- **Laboratory and "1-Off"**
  To meet the goals of quick assembly and easy modification, we favor through-hole prototyping boards, populated with through-hole components, and (where essential) with a small number of SMT devices (on through-hole adapters). This constrains you to *standard logic* and *interface*, a limited selection of *microcontrollers*, and perhaps some *cPLDs/FPGAs*, along with *analog* components (op-amps, etc). You can make greater use of FPGAs if you take advantage of third-party pre-fabricated daughterboards. Alternatively, you can start with a fully populated *microcontroller kit*, including *PLDs*, and perhaps some unpopulated "prototyping" pads for additional devices.
- **Hobbyist**
  Here the idea is to have lots of inexpensive fun. The basic unit is the *microcontroller* (have we mentioned this before?), either as part of a prefabricated *kit*, or used on a through-hole protoboard, or perhaps a custom printed-circuit board. Through-hole components (see *Laboratory and "1-Off", above*) are always easy to use, as are PLDs included in microcontroller kits.